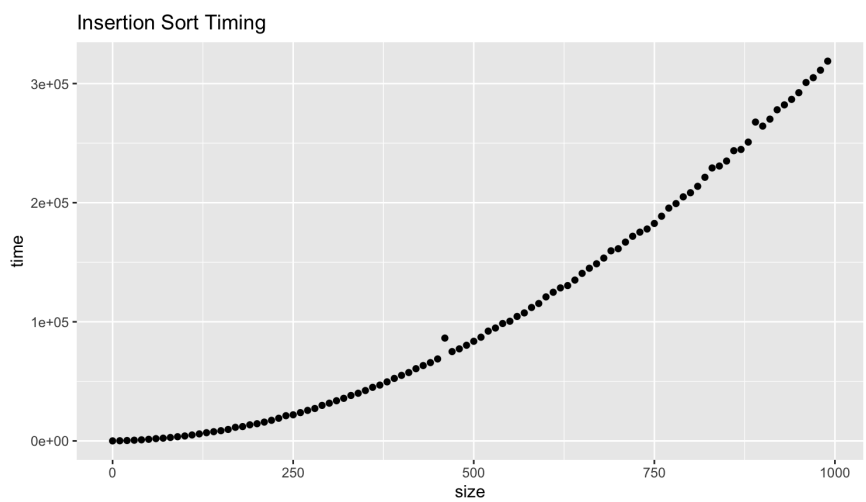
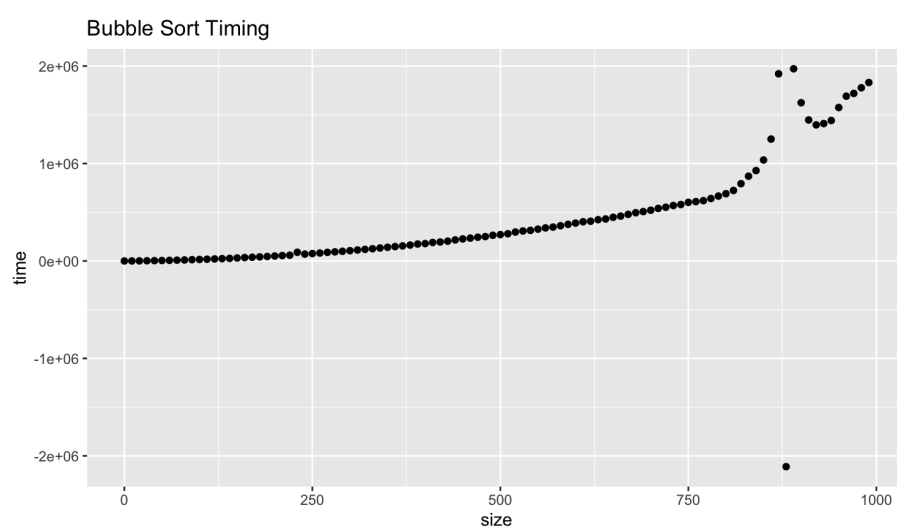
Gwynyn Hayes
Homework #2 More Sorting Algorithms

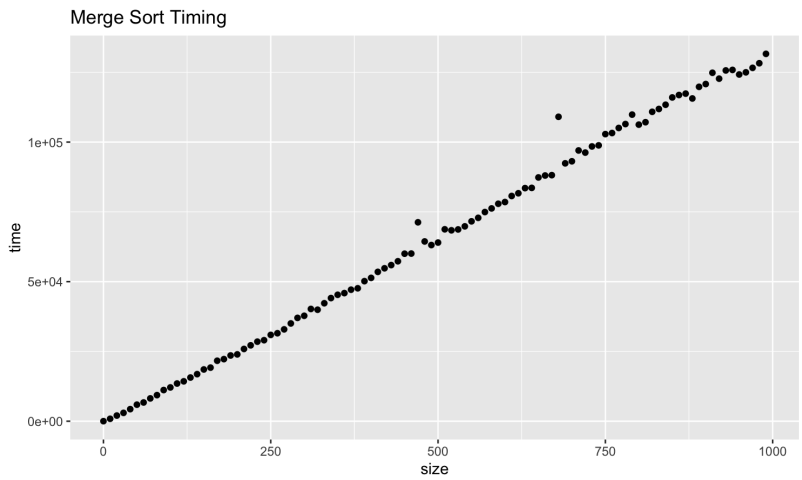
1. In this exercise, you will implement and time insertion sort.
    a. Using the pseudocode for insertion sort provided in class, write an implementation for insertion sort in C++ in a new header file called insertion_sort.h. Insertion sort should be implemented as a function named insertion_sort, which takes an array of integers and an integer giving the size of the array, and returns void. Note that timing_experiment.cpp includes tests to help verify that your implementation works correctly.
    b. Modify timing_experiment.cpp to run a timing experiment on your function insertion_sort. Submit a plot of the results of your timing experiment.



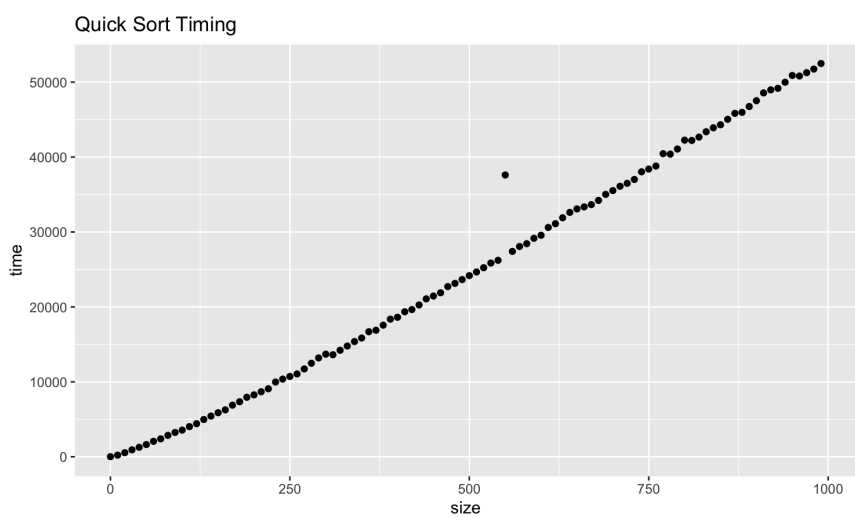
2. In this exercise, you will implement and time bubble sort.
    a. Using the pseudocode for bubble sort provided in class, write an implementation for bubble sort in C++ in a new header file called bubble_sort.h. Bubble sort should be implemented as a function named bubble_sort, which takes an array of integers and an integer giving the size of the array, and returns void.
    b. Modify timing_experiment.cpp to run a timing experiment on your function bubble_sort. Submit a plot of the results of your timing experiment.



3. In this exercise, you will implement and time merge sort.
    a. Using the pseudocode for merge sort provided in class, write an implementation for merge sort in C++ in a new header file called merge_sort.h. Merge sort should be implemented as a function named merge_sort, which takes an array of integers and an integer giving the size of the array, and returns void. Hint: you may find it helpful to define a helper function that takes an array of integers, a start index, and a stop index, and performs merge sort on the array between these indices.
    b. Modify timing_experiment.cpp to run a timing experiment on your function merge_sort. Submit a plot of the results of your timing experiment.

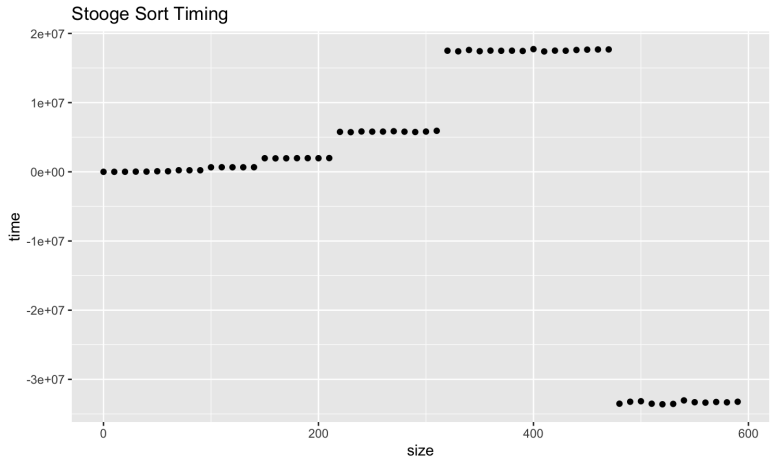


4. In this exercise, you will implement and time quick sort.
    a. Using the pseudocode for quick sort provided in class, write an implementation for quick sort in C++ in a new header file called quick_sort.h. Quick sort should be implemented as a function named quick_sort, which takes an array of integers and an integer giving the size of the array, and returns void. Hint: you may find it helpful to define a helper function that takes an array of integers, a start index, and a stop index, and performs quick sort on the array between these indices.
    b. Modify timing_experiment.cpp to run a timing experiment on your function quick_sort. Submit a plot of the results of your timing experiment.



5. In class, we found that insertion sort and bubble sort have $O(n^2)$ run time, while merge sort and quick sort have $O(n\log(n))$ run time. Based on the results of your timing experiments above, do you think that $O(n^2)$ algorithms or $O(n\log(n))$ algorithms have better time efficiency? Explain your answer.
    a. O(nlog(n)) has a better time efficiency because they are linearly increasing. Whereas insertion and bubble sort have the same time complexity for some of the smaller sized arrays as the bigger arrays. So it's not just the size of the array that is costing the algorithm time but more beyond that.

6. In this exercise, you will implement and time stooge sort, which we describe and provide pseudocode for here. **Stooge Sort**. If the first value is greater than the last value, swap them. Then, recursively sort the first two thirds of the array, and recursively sort the last two thirds of the array, and recursively sort the first two thirds of the array again.
    a. Write an implementation for stooge sort in C++ in a new header file called stooge_sort.h. Stooge sort should be implemented as a function named stooge_sort, which takes an array of integers and an integer giving the size of the array, and returns void.
    b. Modify timing_experiment.cpp to run a timing experiment on your function stooge_sort. Submit a plot of the results of your timing experiment. Note: you will probably need to adjust the maximum size and number of trials to get your timing functions to run in a reasonable amount of time. On my computer, I was able to run this for a maximum size of 600 with 50 trials for each size. I also recommend changing from nanoseconds to microseconds, to avoid overflow errors.



    c. How does the time efficiency of stooge sort compare to our other sorting algorithms? Explain your answer.
        i. Stooge sort's time efficiency is much much worse than any of the other algorithms the smaller the array the longer the algorithm takes but it is still taking much more time then any of the other algorithms

7. (Puzzle Problem) Try to solve the following problem, and submit notes on your problem solving process (e.g., what ideas do you have? What did you try? What works? What doesn't work? etc.). You can earn full credit by demonstrating problem solving effort, even if you do not correctly solve the puzzle.
   You are planning a chess tournament for students in the Computer Science Program. In each round, each student plays another student in chess. The winning student continues to the next round, and the losing student is eliminated from the tournament (assume there are no draws). When there is only one student remaining, they are the winner of the tournament, and they are named the best chess player in the Computer Science Program. Suppose *n* students sign up for the tournament
   a. How many chess matches will be played in the entire tournament?
      i. If there are n students then each student plays another student we have n/2, then we keep doing n/2 until there is a remainder of 1.
   b. How many rounds will there be in the tournament?
      i. Log n rounds in the tournament because if we have 64 people in the tournament to get a single winner we would need 6 rounds which is 2^6 and since we have 2 players competing against each other we have log base 2
   c. To determine the second-best chess player in the program, how many more matches would need to be played?
      i. There would need to be no additional rounds because the second best player is the person that lost to the first best player.